

Πολυτεχνική Σχολή, Πανεπιστήμιο Πατρών
Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Μάθημα: Συστήματα Διαχείρισης Μεγάλων Δεδομένων

Διδάσκοντες: Β.Μεγαλοοικονόμου, Ε.Ζαχαράκη

Υλοποιητικό Project

Εαρινού Εξαμήνου 2026

Όνομα:	Μαρία	Χριστόφορος
Επώνυμο:	Δρούγκα	Θεοδοσόπουλος
Α.Μ.:	1100545	1100557
Έτος/Εξάμηνο:	4ο/8ο	4ο/8ο
Mail:	up1100545@ac.upatras.gr	up1100557@ac.upatras.gr

Πάτρα 2025-2026

Περιεχόμενα

1. Περιγραφή Αρχιτεκτονικής.....σελ3

1.1 Συνολική Ροή Δεδομένων.....σελ3

2. Ερώτημα 1 - Παραγωγή και Αποστολή Δεδομένων σε Redpanda (producer.py).....σελ3

2.1 Περιγραφή.....σελ3

2.2 Εκτέλεση.....σελ3

3. Ερώτημα 2 & 3 – Spark Streaming & MongoDB (spark_streaming.py) σελ4

3.1 Περιγραφή.....σελ4

4. Σχεδιαστικές Επιλογές.....σελ5

4.1 Docker Compose.....σελ5

4.2 Παραμετροποίηση Producer.....σελ5

4.3 OutputMode Update για Stats.....σελ5

4.4 ForeachBatch αντί για Mongo Sink.....σελ5

5. Οδηγίες Εκτέλεσης.....σελ6

5.1 Προαπαιτούμενα.....σελ6

5.2 Αναλυτικά Βήματα Εκτέλεσης.....σελ6

6. Bonus – Extra Υλοποιήσεις.....σελ10

6.1 Windowed Aggregations (spark_streaming_bonus.py) σελ10

6.2 Επιπλέον πεδία Stats.....σελ11

6.3 REST API (api.py) σελ11

6.4 WEB Dashboard.....σελ13

1. Περιγραφή Αρχιτεκτονικής

Το συγκεκριμένο project υλοποιεί ένα ολοκληρωμένο end-to-end pipeline επεξεργασίας ροής δεδομένων in real time, αποτελούμενο από πέντε βασικά πεδία:

1.1 Συνολική Ροή Δεδομένων

UXSIM (Producer) → Redpanda(Broker) → Apache Spark(Consumer) → MongoDB(Storage) → REST API → Dashboard

Πιο αναλυτικά:

- Ο **UXSIM** εξομοιώνει κυκλοφορία οχημάτων σε ένα δίκτυο 4x4 κόμβων για διάστημα 1 ώρας.
  Μας παρέχει την συνάρτηση vehicles_to_pandas(), η οποία επιστρέφει, και θα πάρουμε μελλοντικά εμείς, τα δεδομένα κίνησης όλων των οχημάτων - συγκεκριμένα χρονοσειρές για κάθε όχημα- σε μορφή pandas DataFrame. Επιπλέον, σχετικά με τις στήλες του DataFrame, αυτές που είναι βασικές και θα χρησιμοποιήσουμε στον producer είναι:
  - vehicle_id (μοναδικό αναγνωριστικό οχήματος)
  - link (το τρέχον τμήμα του δρόμου)
  - position (θέση επάνω στον δρόμο/link)
  - time (χρονική στιγμή εξομοίωσης σε secs)
  - speed (ταχύτητα οχήματος m/s)
- Ο **Kafka Producer** (producer.py) λαμβάνει τα data ανά simulation step και τα στέλνει ως JSON αρχεία στο topic vehicle_positions που δημιουργούμε στο Redpanda.
- Το **Redpanda** λειτουργεί ως Kafka-compatible message broker, αποθηκεύοντας προσωρινά τα messages και αποσυνδέοντας producer από consumer.
- Το **Apache Spark** καταναλώνει τα μηνύματα σε micro-batches, επεξεργάζεται και αποθηκεύει τα αποτελέσματα στην MongoDB.
- Η **MongoDB** αποθηκεύει 3 συλλογές: raw_data, stats και windowed_stats(bonus).
- Το **REST API(Flask)** εκθέτει τα data μέσω HTTP endpoints και το dashboard κάνει οπτικοποίηση(visualization) των αποτελεσμάτων in real time για πιο clean και όμορφο αποτέλεσμα.

2. Ερώτημα 1 - Παραγωγή και Αποστολή Δεδομένων σε Redpanda (producer.py)

2.1 Περιγραφή

Το αρχείο producer.py αναλαμβάνει την εξομοίωση της κυκλοφορίας και την αποστολή των δεδομένων στο Redpanda. Καλεί τον UXSIM για νέα δεδομένα. Αρχικά, χρησιμοποιούμε ένα δικό μας αρχείο config.yml για να διαβάσει τις βασικές ρυθμίσεις του Kafka producer. Στην συνάρτηση parse_args() ,ελέγχει αν υπάρχει το config.yml ,φορτώνει τις τιμές interval , topic, broker και ενημερώνει τις default τιμές με αυτές του αρχείου. Έτσι, επιτρέπουμε την εύκολη αλλαγή του Kafka topic και την αλλαγή διεύθυνσης του broker. Στην συνέχεια γίνεται εξαγωγή δεδομένων σε DataFrame (με στήλες name, dn, orig, dest, t, link, x, s, v) φιλτράροντας με την εντολή: **df_moving_vehicles = df[df["v"] > 0].copy()** μόνο τα κινούμενα οχήματα με speed>0 .

2.2 Εκτέλεση

Μέσα σε cmd/Windows Powershell τρέχουμε:

```
python3 producer.py
```

3. Ερώτημα 2 & 3 – Spark Streaming & MongoDB (spark_streaming.py)

3.1 Περιγραφή

Το αρχείο spark_streaming.py υλοποιεί τον Spark Streaming consumer, ο οποίος καταναλώνει δεδομένα από το Redpanda και έπειτα επεξεργάζεται και τα αποθηκεύει στην βάση μας MongoDB.

Spark Session

Η σύνδεση στην MongoDB γίνεται μέσω του MongoDB Spark Connector με URI mongodb://mongo:27017. Τα απαραίτητα JARs φορτώνονται κατά την εκκίνηση (Kafka connector, MongoDB connector).

Stream Reading & Parsing

Το Spark συνδέεται στο Redpanda μέσω του Kafka API και διαβάζει από το topic vehicle_positions, με startingOffsets=latest (δηλαδή καταναλώνει μόνο νέα μηνύματα). Τα binary messages μετατρέπονται σε strings και στην συνέχεια ως JSON με βάση το παρακάτω σχήμα:

```
StructType: name(string), dn(int), orig(string), dest(string), time(double), link(string), x(double), s(double), v(double)
```

Στατιστικά ανά ακμή (Ερώτημα 2)

Για κάθε micro-batch που λαμβάνει η Spark, κάνουμε υπολογισμό aggregations με groubBy(time, link):

- vcount: πλήθος οχημάτων ανά ακμή
- vspeed: μέση ταχύτητα οχήματων
- average_spacing: μέση απόσταση μεταξύ των οχημάτων (bonus)
- min_speed/max_speed: ελάχιστη/μέγιστη ταχύτητα (bonus)

Αποθήκευση στην MongoDB (Ερώτημα 3)

Χρησιμοποιούμε το foreachBatch API του Spark για εγγραφή σε 2 συλλογές:

- traffic.raw_data: Οι raw εγγραφές δεδομένων των οχημάτων με outputMode append
- traffic.stats: Τα επεξεργασμένα στατιστικά δεδομένα των οχημάτων με outputMode update, το οποίο μας εξασγαλίζει ότι γράφονται μόνο οι ακμές που άλλαξαν στο τρέχον micro-batch, μειώνοντας τον όγκο των εγγραφών μας.
- $+3^n$ συλλογή windowed_stats για bonus

4. Σχεδιαστικές Επιλογές

4.1 Docker Compose

Η υποδομή (Redpanda, Spark, MongoDB) εκτελείται μέσω Docker Compose.Το docker-compose.yml ορίζει 5 υπηρεσίες που επικοινωνούν μέσω του κοινού Docker network dsnet:

- Spark-master
- Spark-worker
- Spark-submit-job
- Redpanda
- mongodb

4.2 Παραμετροποίηση Producer

Ο producer υποστηρίζει 3 επίπεδα παραμετροποίησης(defaults->config.yml->CLI),ώστε να είναι εύκολα προσαρμόσιμος χωρίς αλλαγές στον κώδικα. Αυτό επιτρέπει π.χ εύκολη αλλαγή του interval για testing.

4.3 OutputMode Update για Stats

Στο stats sink χρησιμοποιείται outputMode(“update”) αντί append,ώστε το Spark να εγγράφει στην MongoDB μόνο τις ακμές που άλλαξαν στο τρέχον micro-batch, μειώνοντας τον όγκο των εγγραφών.

4.4 ForeachBatch αντί για Mongo Sink

Η χρήση foreachBatch αντί του ενσωματωμένου MongoDB sink δίνει πλήρη έλεγχο στην λογική εγγραφής και επιτρέπει να χρησιμοποιηθεί ο standard DataFrame writer.

5. Οδηγίες Εκτέλεσης

5.1 Προαπαιτούμενα

- ✓ Docker Desktop εγκατεστημένο και σε λειτουργία
- ✓ Python 3.x με εγκατεστημένα:
  1. Uxsim
  2. Kafka-python
  3. Pyyaml
  4. Flask-cors
- ✓ WSL2 με Ubuntu (για Window users)

5.2 Αναλυτικά Βήματα Εκτέλεσης

Βήμα 1 – Εκκίνηση υποδομής (Redpanda + Spark + MongoDB)

Αρχικά, ανοίγουμε το WSL/Ubuntu, μπαίνουμε στον φάκελο που έχουμε τους κώδικές μας και εκτελούμε τις παρακάτω εντολές:

a) Διαγραφή των παλιών docker:

```
docker-compose down
```

b) Εγκατάσταση και run:

```
docker-compose up –d redpanda
```

Πρώτα ενεργοποιούμε το Redpanda.

```
docker exec redpanda rpk topic create vehicle_positions
```

Δημιουργούμε το topic μας.

```
docker exec redpanda rpk topic list
```

Ελέγχουμε αν το topic μας έχει δημιουργηθεί και υπάρχει στην λίστα.

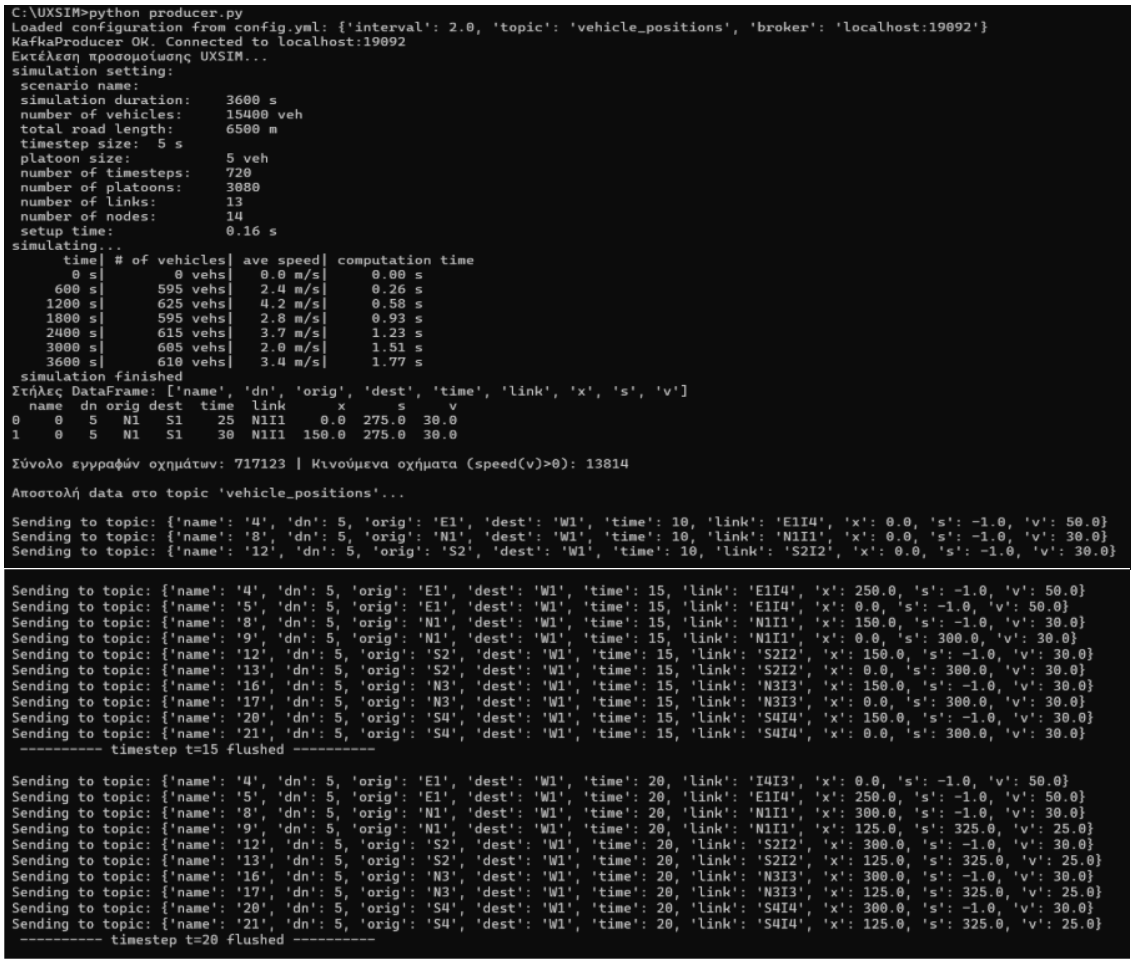
```
docker-compose up –d
```

```
xrist@MSI:~/Big_Data$ docker-compose down
Stopping spark-worker ... done
Stopping mongo        ... done
Stopping spark-master ... done
Stopping redpanda     ... done
Removing spark-job    ... done
Removing spark-worker ... done
Removing mongo        ... done
Removing spark-master ... done
Removing redpanda     ... done
Removing network big_data_dsnet
xrist@MSI:~/Big_Data$ docker-compose up -d redpanda
Creating network "big_data_dsnet" with driver "bridge"
Creating redpanda ... done
xrist@MSI:~/Big_Data$ docker exec redpanda rpk topic create vehicle_positions
TOPIC             STATUS
vehicle_positions  OK
xrist@MSI:~/Big_Data$ docker exec redpanda rpk topic list
NAME              PARTITIONS  REPLICAS
vehicle_positions  1           1
xrist@MSI:~/Big_Data$ docker-compose up -d
redpanda is up-to-date
Creating spark-master ... done
Creating mongo        ... done
Creating spark-worker ... done
Creating spark-job    ... done
xrist@MSI:~/Big_Data$
```

Βήμα 2 – Ενεργοποίηση του UXSIM (producer)

Ανοίγουμε cmd/Windows Powershell, μπαίνουμε στον φάκελο που έχουμε τους κώδικές μας και και εκτελούμε:

```
python producer.py
```

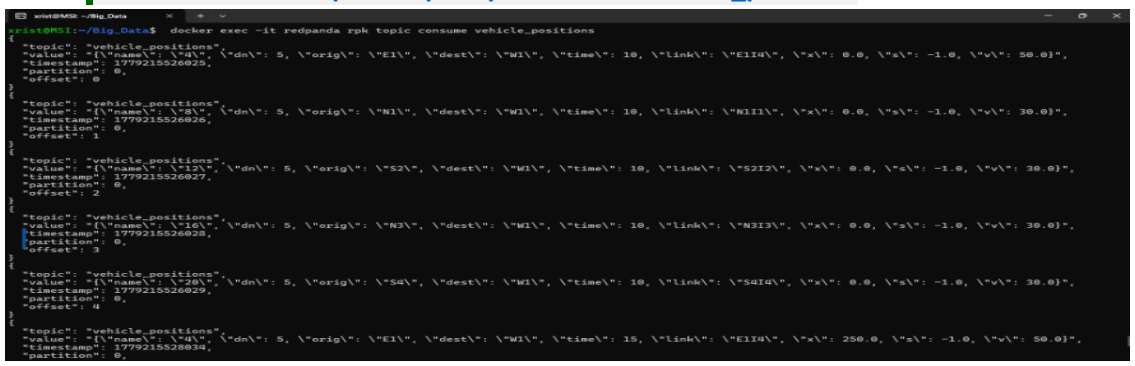


Μέσα από το παραπάνω screenshot εκτέλεσης παρατηρούμε ότι έχουμε 717.123 εγγραφές οχημάτων, ενώ μετά το φιλτράρισμα με speed>0 εμείς κάνουμε εξαγωγή 13.814 εγγραφές μόνο με τα κινούμενα οχήματα.

Βήμα 3 – Live Αποτελέσματα

Ανοίγουμε ξανά το WLS/Ubuntu, και για να δούμε live τα αποτελέσματα της παραπάνω εντολής τρέχουμε:

```
docker exec -it redpanda rpk topic consume vehicle_positions
```





Βήμα 4 – Ενεργοποίηση Spark Streaming

Μέσα στο WSL/Ubuntu ,για να τρέξουμε το spark_streaming.py αρχικά δημιουργούμε ένα copy του spark_streaming.py μέσα στο container:

```
docker cp spark_streaming.py spark-master:/opt/spark/work-dir/
```

Έπειτα, τρέχουμε την Spark Streaming εφαρμογή του spark_streaming.py . Μέσα στο WSL/Ubuntu μπαίνουμε μέσα στο container του spark-master με την εντολή:

```
docker exec -it spark-master bash
```

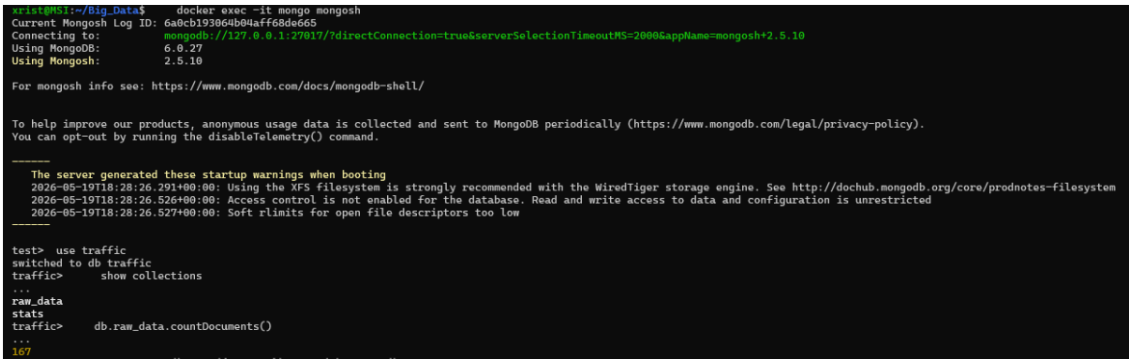
Μέσα στο container τρέχουμε την εξής εντολή (ταυτόχρονα με τον producer.py):

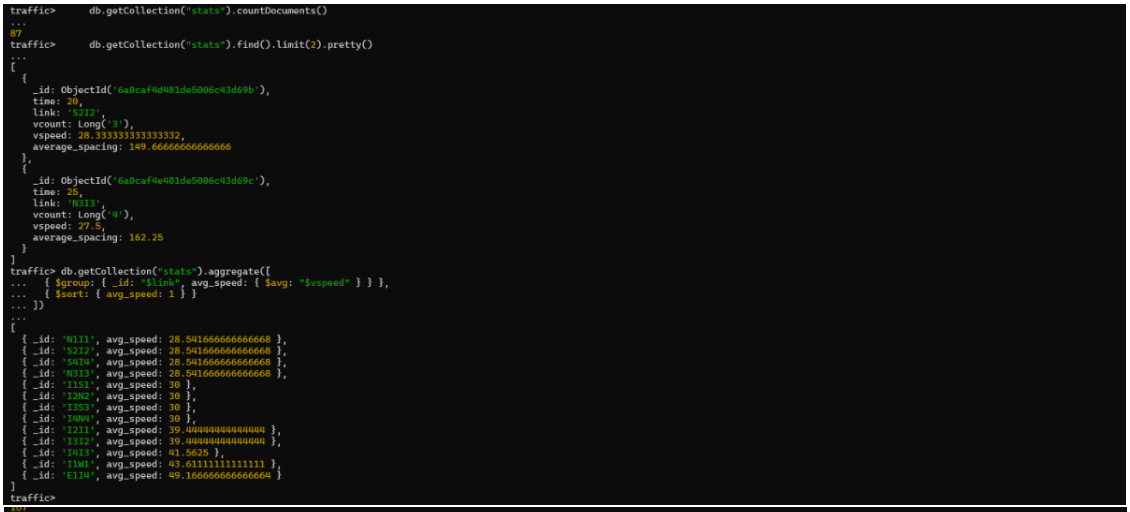
```
/opt/spark/bin/spark-submit /opt/spark/work-dir/spark_streaming.py
```

Βήμα 5 – Ενεργοποίηση και έλεγχος MongoDB

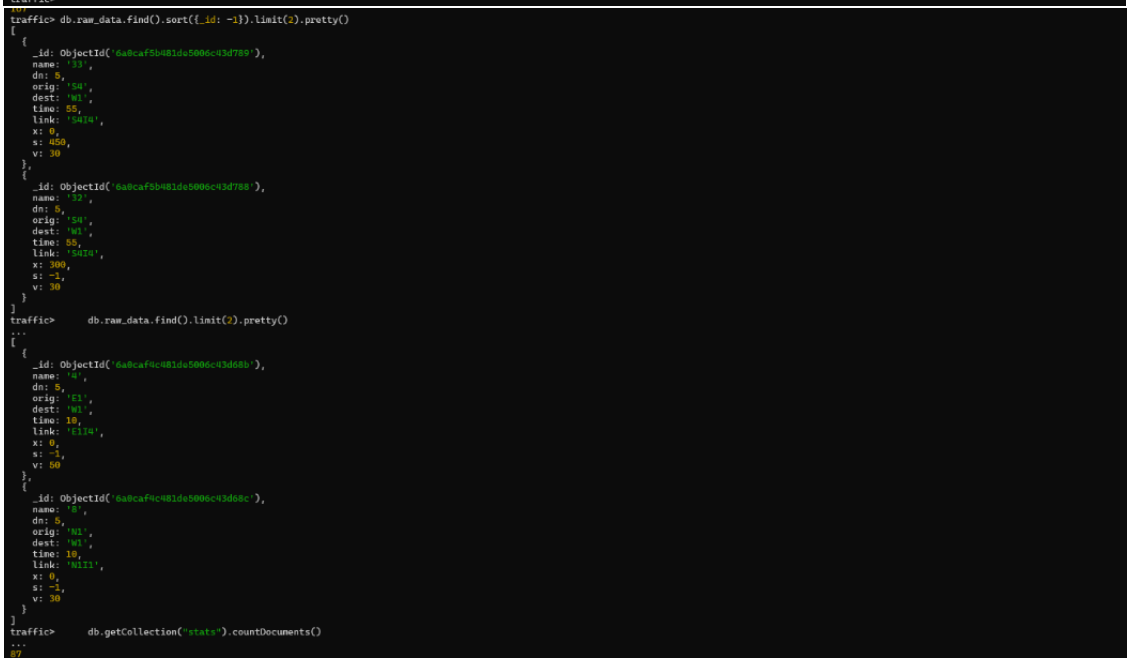
Μέσα στο WSL/Ubuntu, μπαίνουμε στην MongoDB, μέσω του mongosh, και τρέχουμε τις εξής εντολές:

```
docker exec -it mongo mongosh
use traffic // επιλογή database
show collections
db.raw_data.countDocuments()
db.raw_data.find().limit(2).pretty()
db.getCollection("stats").countDocuments()
db.getCollection("stats").find().limit(2).pretty()
```





```
traffic>      db.getCollection("stats").countDocuments()
...
87
traffic>      db.getCollection("stats").find().limit(2).pretty()
...
[
  {
    _id: ObjectId('6a0caf4d481de5006c43d69b'),
    time: 20,
    link: 'S2I2',
    vcount: Long('3'),
    vspeed: 28.333333333333332,
    average_spacing: 149.66666666666666
  },
  {
    _id: ObjectId('6a0caf4e481de5006c43d69c'),
    time: 25,
    link: 'N3I3',
    vcount: Long('4'),
    vspeed: 27.5,
    average_spacing: 162.25
  }
]
traffic> db.getCollection("stats").aggregate([
...   { $group: { _id: "$link", avg_speed: { $avg: "$vspeed" } } },
...   { $sort: { avg_speed: 1 } }
... ])
...
[
  { _id: 'N1I1', avg_speed: 28.541666666666668 },
  { _id: 'S2I2', avg_speed: 28.541666666666668 },
  { _id: 'S4I4', avg_speed: 28.541666666666668 },
  { _id: 'N3I3', avg_speed: 28.541666666666668 },
  { _id: 'I1S1', avg_speed: 30 },
  { _id: 'I2N2', avg_speed: 30 },
  { _id: 'I3S3', avg_speed: 30 },
  { _id: 'I4N4', avg_speed: 30 },
  { _id: 'I2I1', avg_speed: 39.444444444444444 },
  { _id: 'I3I2', avg_speed: 39.444444444444444 },
  { _id: 'I4I3', avg_speed: 41.5625 },
  { _id: 'I1W1', avg_speed: 43.611111111111111 },
  { _id: 'E1I4', avg_speed: 49.166666666666664 }
]
traffic>
```



```
traffic> db.raw_data.find().sort({_id: -1}).limit(2).pretty()
[
  {
    _id: ObjectId('6a0caf5b481de5006c43d789'),
    name: '33',
    dn: 5,
    orig: 'S4',
    dest: 'W1',
    time: 55,
    link: 'S4I4',
    x: 0,
    s: 450,
    v: 30
  },
  {
    _id: ObjectId('6a0caf5b481de5006c43d788'),
    name: '32',
    dn: 5,
    orig: 'S4',
    dest: 'W1',
    time: 55,
    link: 'S4I4',
    x: 300,
    s: -1,
    v: 30
  }
]
traffic>      db.raw_data.find().limit(2).pretty()
...
[
  {
    _id: ObjectId('6a0caf4c481de5006c43d68b'),
    name: '4',
    dn: 5,
    orig: 'E1',
    dest: 'W1',
    time: 10,
    link: 'E1I4',
    x: 0,
    s: -1,
    v: 50
  },
  {
    _id: ObjectId('6a0caf4c481de5006c43d68c'),
    name: '5',
    dn: 5,
    orig: 'N1',
    dest: 'W1',
    time: 10,
    link: 'N1I1',
    x: 0,
    s: -1,
    v: 30
  }
]
traffic>      db.getCollection("stats").countDocuments()
...
87
```

*Note: Τρέξαμε db.raw_data.find().sort({_id: -1}).limit(2),pretty() για να μας δείξει τα τελευταία data, επειδή σε αυτό στο session όταν βγάλαμε τα screenshots μας έβγαζε τις πρώτες εγγραφές με null

Έπειτα, χρειάζεται και έλεγχος aggregate query που τρέχουμε στο mongosh, για να μας δείξει τη μέση ταχύτητα ανά link, ταξινομημένη από την πιο αργή (πιο συμφορημένη) στην πιο γρήγορη:

```
db.getCollection("stats").aggregate([
        { $group: { _id: "$link", avg_speed: { $avg: "$vspeed" } } },
        { $sort: { avg_speed: 1 } }
        ])
```



```
traffic> db.getCollection("stats").aggregate([
...   { $group: { _id: "$link", avg_speed: { $avg: "$vspeed" } } },
...   { $sort: { avg_speed: 1 } }
... ])
...
[
  { _id: 'N1I1', avg_speed: 28.068181818181817 },
  { _id: 'N3I3', avg_speed: 28.068181818181817 },
  { _id: 'S4I4', avg_speed: 28.068181818181817 },
  { _id: 'S2I2', avg_speed: 28.068181818181817 },
  { _id: 'I3S3', avg_speed: 30 },
  { _id: 'I1S1', avg_speed: 30 },
  { _id: 'I4N4', avg_speed: 30 },
  { _id: 'I2N2', avg_speed: 30 },
  { _id: 'I2I1', avg_speed: 40.238095238095234 },
  { _id: 'I3I2', avg_speed: 40.238095238095234 },
  { _id: 'I4I3', avg_speed: 40.238095238095234 },
  { _id: 'I1W1', avg_speed: 46.42857142857143 },
  { _id: 'E1I4', avg_speed: 47.27272727272727 }
]
traffic>
```

Βήμα 6 (Bonus) – Εκκίνηση REST API

Το REST API το τρέχουμε σε virtual environment (venv) που έχουμε δημιουργήσει στον υπολογιστή μας (πχ venv_project). Για να το εκκινήσουμε τρέχουμε τις εξής εντολές:

```
source  ~/venv_project/bin/activate
```

Σε αυτό το σημείο, πρέπει να κάνουμε (αν ήδη δεν έχουμε κάνει):

```
pip install flask-cors
```

```
python3 api.py
```

Βήμα 7 – Εκκίνηση web server για το dashboard visualization

Για την εκκίνηση του web server, χρησιμοποιήσαμε ένα εναλλακτικό port (8181) για τον http.server, επειδή το port 8080 ήταν ήδη σε χρήση από τον Spark UI. (Μπαίνουμε σε νέο Ubuntu και στον φάκελο …/visuals).

```
python3 –m http.server 8181
```

Και για να ανοίξουμε το dashboard στον browser:

```
http://localhost:8181/traffic_dashboard_visualization.html
```

6. Bonus – Extra Υλοποιήσεις

6.1 Windowed Aggregations (spark_streaming_bonus.py)

Στο συγκεκριμένο python script υλοποιήσαμε windowed aggregations με παράθυρα των 30 δευτερολέπτων. Σε αυτό μας βοήθησε το simulation time της UXSIM που , ουσιαστικά, χρησιμοποιήθηκε ως event timer. Η λογική και προσέγγισή μας ήταν η εξής:

- Αρχικά, μετατρέπουμε το πεδίο του time(χρόνου) (float seconds) σε Spark Timestamp. Ήταν μια βασική πρόκληση καθώς το Spark αναμένει πραγματικά timestamps για windowed operations, ενώ το πεδίο της UXSIM είναι απλά ένας αριθμός seconds(float).
  Η λύση που βρήκαμε ήταν να μετατρέψουμε το simulation time σε Spark timestamp προσθέτοντάς το στο Unix epoch (1970-01-01) με την χρήση του **to_timestamp(col(“time”).cast(“long”))**
- Έπειτα, χρειάστηκε εφαρμογή **Watermark 60 δευτερολέπτων** για τον χειρισμό late-arriving data
- GroupBy με **window(event_time, “30 seconds”)** και **link**
- Και τέλος αποθήκευση στην συλλογή **traffic.windowed_stats**

Επίσης, κάθε εγγραφή windowed_stats περιέχει:

- window_start
- window_end
- link
- vcount
- vspeed
- average_spacing
- min_speed
- max_speed

6.2 Επιπλέον πεδία Stats

Πέραν των βασικών απαιτήσεων (vcount, vspeed), προστέθηκαν, με βάση και τις απαιτήσεις της bonus υλοποίησης στην αναφορά:

- **min_speed:** Η ελάχιστη ταχύτητα οχήματος στην ακμή (spark_min("v"))
- **max_speed:** Η μέγιστη ταχύτητα οχήματος στην ακμή (spark_max("v"))
- **average_spacing:** Η μέση απόσταση μεταξύ των οχημάτων (avg("s"))

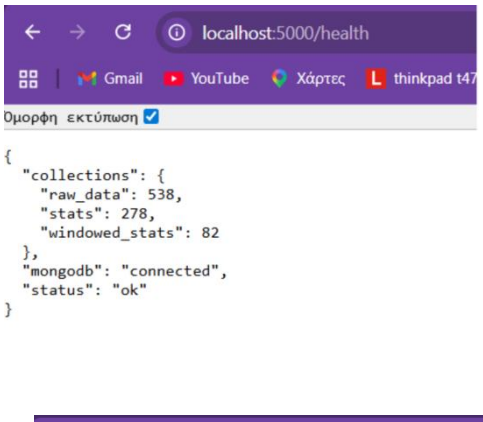
Note/Παρατηρήσεις:
Η τιμή **average_spacing=-1.0** δείχνει ότι δεν μπορεί να υπολογιστεί ουσιαστικά μέση απόσταση .Αυτό συμβαίνει όταν στο συγκεκριμένο link και στην συγκεκριμένη χρονική στιγμή υπάρχει μόνο ένα όχημα, άρα δεν υπάρχει 2ο όχημα για να υπολογιστεί απόσταση μεταξύ τους.

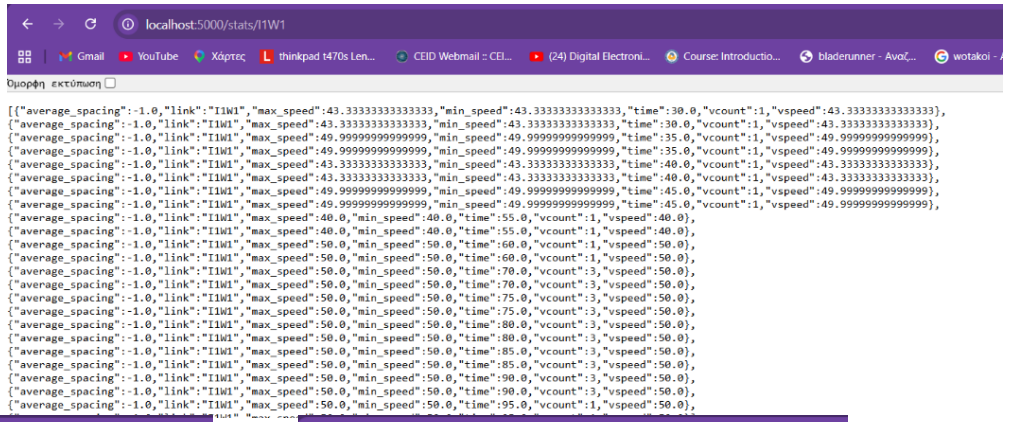
6.3 REST API (api.py)

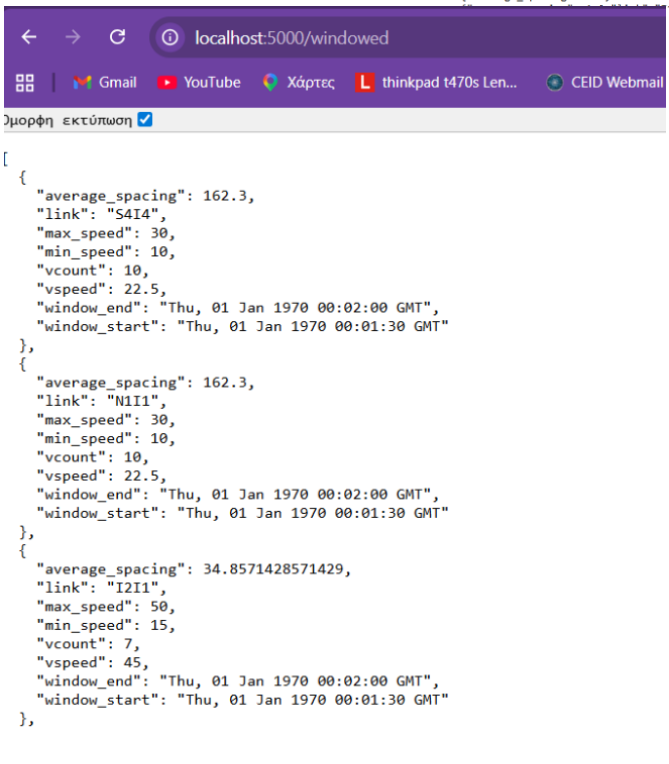
Υλοποιήσαμε, επίσης, ένα πλήρες REST API με Flask που εκθέτει 6 βασικά endpoints:

ENDPOINT	ΠΕΡΙΓΡΑΦΗ
GET/health	Έλεγχος σύνδεσης με την MongoDB και επίσης βλέπουμε το πλήθος των εγγραφών ανά συλλογή.
GET/stats	Βλέπουμε τα latest statistics ανά ακμή (MongoDB aggregation pipeline, για να επιστρέφει πάντα την πιο πρόσφατη εγγραφή ανά ακμή).
GET/stats/<link>	Βλέπουμε ιστορικό των δεδομένων για συγκεκριμένη ακμή.
GET/windowed	Windowed aggregations με optional filter ανά link.
GET/raw	Raw εγγραφές οχημάτων.
GET/summary	Γενικό, συγκεντρωτικό summary στατιστικών για όλο το διάστημα της προσομοίωσής μας.

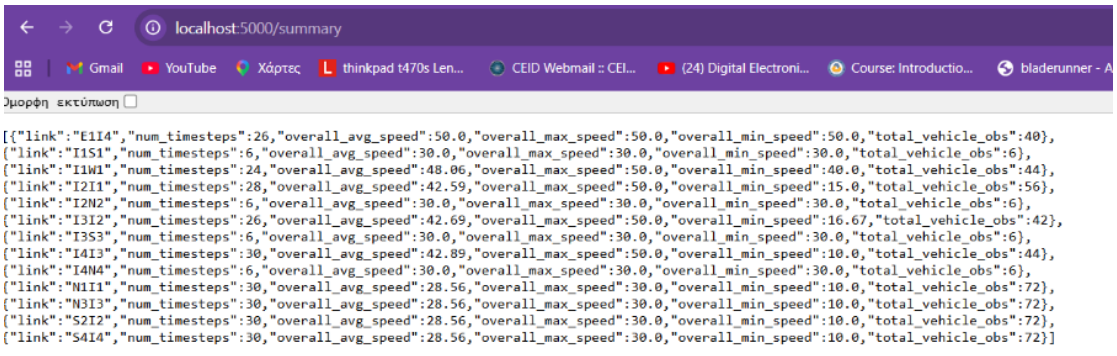
Επίσης, για να επιτρέψουμε την πρόσβαση από τον browser dashboard, προσθέσαμε CORS support μέσω της βιβλιοθήκης flask-cors.


localhost:5000/health
Gmail
YouTube
Χάρτες
thinkpad t47
Όμορφη εκτύπωση
{
"collections": {
"raw_data": 538,
"stats": 278,
"windowed_stats": 82
},
"mongodb": "connected",
"status": "ok"
}


localhost:5000/stats/I1W1
Gmail
YouTube
Χάρτες
thinkpad t470s Len...
CEID Webmail :: CEI...
(24) Digital Electroni...
Course: Introductio...
bladerunner - Αναζ...
Όμορφη εκτύπωση
[{"average_spacing":-1.0,"link":"I1W1","max_speed":43.33333333333333,"min_speed":43.33333333333333,"time":30.0,"vcount":1,"vspeed":43.33333333333333},
{"average_spacing":-1.0,"link":"I1W1","max_speed":43.33333333333333,"min_speed":43.33333333333333,"time":30.0,"vcount":1,"vspeed":43.33333333333333},
{"average_spacing":-1.0,"link":"I1W1","max_speed":49.99999999999999,"min_speed":49.99999999999999,"time":35.0,"vcount":1,"vspeed":49.99999999999999},
{"average_spacing":-1.0,"link":"I1W1","max_speed":49.99999999999999,"min_speed":49.99999999999999,"time":35.0,"vcount":1,"vspeed":49.99999999999999},
{"average_spacing":-1.0,"link":"I1W1","max_speed":43.33333333333333,"min_speed":43.33333333333333,"time":40.0,"vcount":1,"vspeed":43.33333333333333},
{"average_spacing":-1.0,"link":"I1W1","max_speed":43.33333333333333,"min_speed":43.33333333333333,"time":40.0,"vcount":1,"vspeed":43.33333333333333},
{"average_spacing":-1.0,"link":"I1W1","max_speed":49.99999999999999,"min_speed":49.99999999999999,"time":45.0,"vcount":1,"vspeed":49.99999999999999},
{"average_spacing":-1.0,"link":"I1W1","max_speed":49.99999999999999,"min_speed":49.99999999999999,"time":45.0,"vcount":1,"vspeed":49.99999999999999},
{"average_spacing":-1.0,"link":"I1W1","max_speed":40.0,"min_speed":40.0,"time":55.0,"vcount":1,"vspeed":40.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":40.0,"min_speed":40.0,"time":55.0,"vcount":1,"vspeed":40.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":60.0,"vcount":1,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":60.0,"vcount":1,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":70.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":70.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":75.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":75.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":80.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":80.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":85.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":85.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":90.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":90.0,"vcount":3,"vspeed":50.0},
{"average_spacing":-1.0,"link":"I1W1","max_speed":50.0,"min_speed":50.0,"time":95.0,"vcount":1,"vspeed":50.0},


localhost:5000/windowed
Gmail
YouTube
Χάρτες
thinkpad t470s Len...
CEID Webmail
Όμορφη εκτύπωση
[
{
"average_spacing": 162.3,
"link": "S4I4",
"max_speed": 30,
"min_speed": 10,
"vcount": 10,
"vspeed": 22.5,
"window_end": "Thu, 01 Jan 1970 00:02:00 GMT",
"window_start": "Thu, 01 Jan 1970 00:01:30 GMT"
},
{
"average_spacing": 162.3,
"link": "N1I1",
"max_speed": 30,
"min_speed": 10,
"vcount": 10,
"vspeed": 22.5,
"window_end": "Thu, 01 Jan 1970 00:02:00 GMT",
"window_start": "Thu, 01 Jan 1970 00:01:30 GMT"
},
{
"average_spacing": 34.8571428571429,
"link": "I2I1",
"max_speed": 50,
"min_speed": 15,
"vcount": 7,
"vspeed": 45,
"window_end": "Thu, 01 Jan 1970 00:02:00 GMT",
"window_start": "Thu, 01 Jan 1970 00:01:30 GMT"
},


localhost:5000/raw
Gmail
YouTube
Χάρτες
thinkpad t470s Len...
Όμορφη εκτύπωση
[
{
"dest": "W1",
"dn": 5,
"link": "N1I1",
"name": "7",
"orig": "N1",
"s": -1,
"time": 10,
"v": 30,
"x": 0
},
{
"dest": "W1",
"dn": 5,
"link": "N3I3",
"name": "13",
"orig": "N3",
"s": -1,
"time": 10,
"v": 30,
"x": 0
},
{
"dest": "W1",
"dn": 5,
"link": "S2I2",
"name": "10",
"orig": "S2",
"s": -1,
"time": 10,
"v": 30,
"x": 0
},

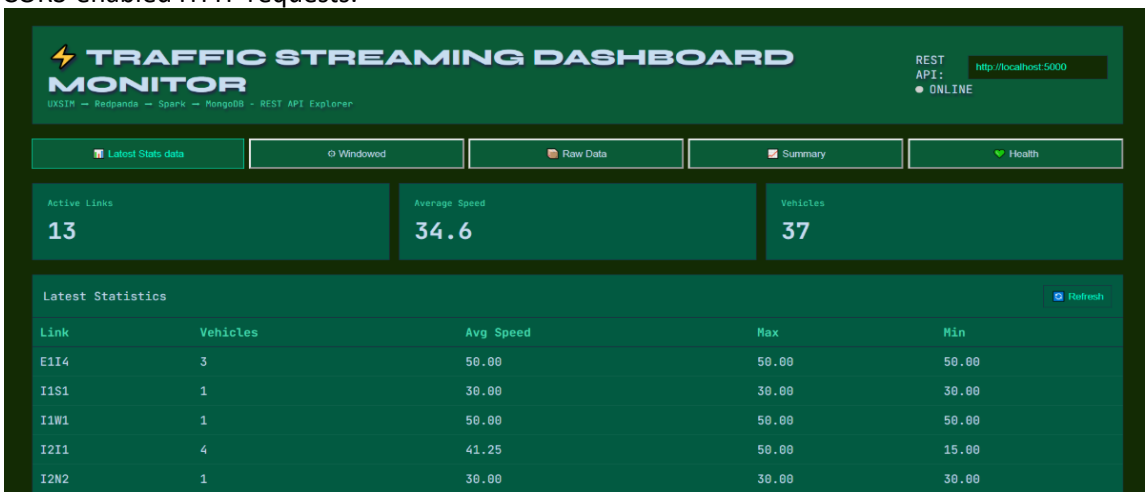

localhost:5000/summary
Gmail
YouTube
Χάρτες
thinkpad t470s Len...
CEID Webmail :: CEI...
(24) Digital Electroni...
Course: Introductio...
bladerunner - A
Όμορφη εκτύπωση
[{"link":"E1I4","num_timesteps":26,"overall_avg_speed":50.0,"overall_max_speed":50.0,"overall_min_speed":50.0,"total_vehicle_obs":40},
{"link":"I1S1","num_timesteps":6,"overall_avg_speed":30.0,"overall_max_speed":30.0,"overall_min_speed":30.0,"total_vehicle_obs":6},
{"link":"I1W1","num_timesteps":24,"overall_avg_speed":48.06,"overall_max_speed":50.0,"overall_min_speed":40.0,"total_vehicle_obs":44},
{"link":"I2I1","num_timesteps":28,"overall_avg_speed":42.59,"overall_max_speed":50.0,"overall_min_speed":15.0,"total_vehicle_obs":56},
{"link":"I2N2","num_timesteps":6,"overall_avg_speed":30.0,"overall_max_speed":30.0,"overall_min_speed":30.0,"total_vehicle_obs":6},
{"link":"I3I2","num_timesteps":26,"overall_avg_speed":42.69,"overall_max_speed":50.0,"overall_min_speed":16.67,"total_vehicle_obs":42},
{"link":"I3S3","num_timesteps":6,"overall_avg_speed":30.0,"overall_max_speed":30.0,"overall_min_speed":30.0,"total_vehicle_obs":6},
{"link":"I4I3","num_timesteps":30,"overall_avg_speed":42.89,"overall_max_speed":50.0,"overall_min_speed":10.0,"total_vehicle_obs":44},
{"link":"I4N4","num_timesteps":6,"overall_avg_speed":30.0,"overall_max_speed":30.0,"overall_min_speed":30.0,"total_vehicle_obs":6},
{"link":"N1I1","num_timesteps":30,"overall_avg_speed":28.56,"overall_max_speed":30.0,"overall_min_speed":10.0,"total_vehicle_obs":72},
{"link":"N3I3","num_timesteps":30,"overall_avg_speed":28.56,"overall_max_speed":30.0,"overall_min_speed":10.0,"total_vehicle_obs":72},
{"link":"S2I2","num_timesteps":30,"overall_avg_speed":28.56,"overall_max_speed":30.0,"overall_min_speed":10.0,"total_vehicle_obs":72},
{"link":"S4I4","num_timesteps":30,"overall_avg_speed":28.56,"overall_max_speed":30.0,"overall_min_speed":10.0,"total_vehicle_obs":72}]

6.4 WEB Dashboard

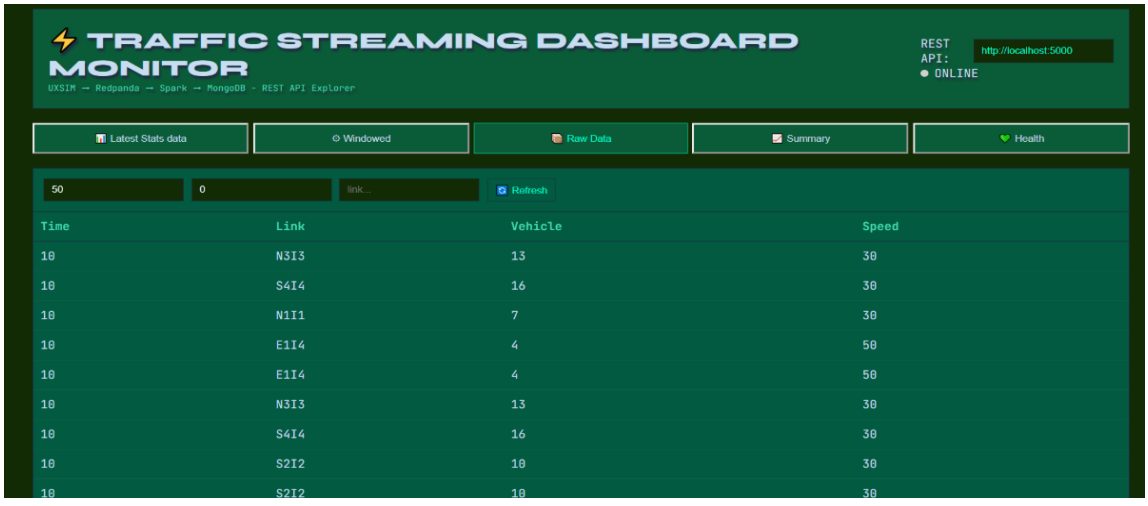
Επιπλέον, σαν extra υλοποίηση, δημιουργήσαμε και ένα διαδραστικό web dashboard (HTML/CSS/JS), που καλεί το REST API και εμφανίζει:

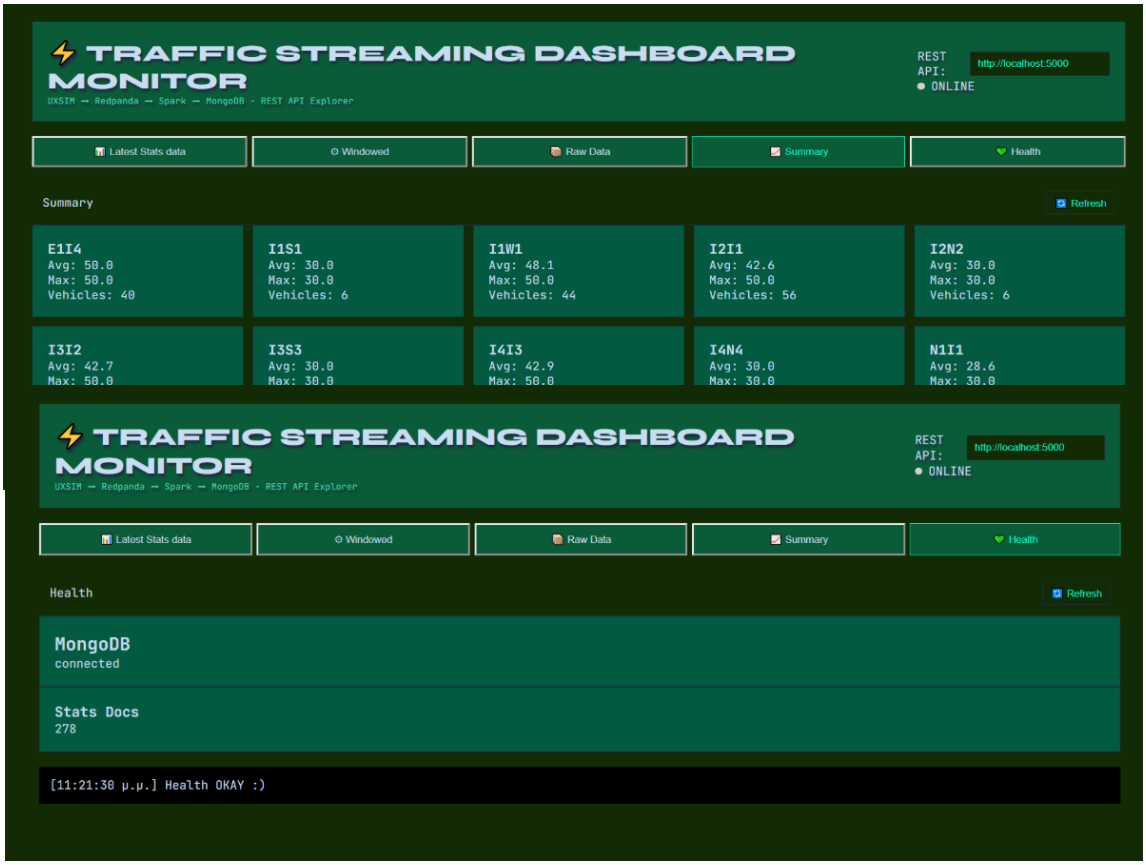
- **KPI cards:** Αριθμός ενεργών ακμών, μέση ταχύτητα και σύνολο οχημάτων
- **Πίνακα Latest Stats** με speed bars ανά ακμή
- **Windowed Statistics** με φίλτρο ανά ακμή
- **Raw data viewer**
- **Health panel** με κατάσταση MongoDB

Το dashboard εκτελείται μέσω python3 –m http.server και επικοινωνεί με το Flask API μέσω CORS-enabled HTTP requests.








TRAFFIC STREAMING DASHBOARD MONITOR
UXSIM → Redpanda → Spark → MongoDB - REST API Explorer
REST API:
http://localhost:5000
ONLINE
Latest Stats data
Windowed
Raw Data
Summary
Health
Summary
Refresh
E1I4
Avg: 50.0
Max: 50.0
Vehicles: 40
I1S1
Avg: 30.0
Max: 30.0
Vehicles: 6
I1W1
Avg: 48.1
Max: 50.0
Vehicles: 44
I2I1
Avg: 42.6
Max: 50.0
Vehicles: 56
I2N2
Avg: 30.0
Max: 30.0
Vehicles: 6
I3I2
Avg: 42.7
Max: 50.0
I3S3
Avg: 30.0
Max: 30.0
I4I3
Avg: 42.9
Max: 50.0
I4N4
Avg: 30.0
Max: 30.0
N1I1
Avg: 28.6
Max: 30.0
TRAFFIC STREAMING DASHBOARD MONITOR
UXSIM → Redpanda → Spark → MongoDB - REST API Explorer
REST API:
http://localhost:5000
ONLINE
Latest Stats data
Windowed
Raw Data
Summary
Health
Health
Refresh
MongoDB
connected
Stats Docs
278
[11:21:30 μ.μ.] Health OKAY :)